OdiaGPT: Pretraining an Indic Decoder-Only Transformer from Scratch for Low-Resource Odia Language Modeling

Himanshu Mohanty
Independent AI Researcher / CodeHima • codehimanshu24@gmail.com
Repository: github.com/HimanshuMohanty-Git24/OdiaGPT • Hugging Face: CodeHima/OdiaGPT-30M

ABSTRACT — Autoregressive large language models (LLMs) have achieved remarkable linguistic and computational capabilities. However, low-resource and non-Latin languages—specifically classical Indic languages such as **Odia** (ଓଡ଼ିଆ), spoken by over 50 million people—remain severely disadvantaged. Commercial and frontier open-weight models suffer from prohibitive "tokenizer taxation" (excessive subword fragmentation), high inference latency, and negligible representation in pretraining corpora ($< 0.01\%$). In this paper, we present **OdiaGPT**, an educational and foundational decoder-only Transformer built and trained **100% from scratch** for the Odia language. Operating under a strict zero-pretrained-dependency policy, OdiaGPT utilizes zero pretrained model checkpoints, zero off-the-shelf multilingual tokenizers, and zero high-level wrapper abstractions. We construct a specialized from-scratch 8,192-vocabulary SentencePiece Byte-Pair Encoding (BPE) tokenizer with 100% character coverage and byte fallback, eliminating character dropping and optimizing token compression for Brahmic scripts. Our flagship model, **OdiaGPT-30M** (31,465,984 parameters), incorporates Rotary Position Embeddings (RoPE), Pre-RMSNorm, SwiGLU feed-forward networks, causal multi-head self-attention with PyTorch Scaled Dot-Product Attention (SDPA), and tied token embeddings. Trained on a single NVIDIA Tesla T4 GPU over 29.5 million tokens from a filtered, NFC-normalized partition of AI4Bharat Sangraha, OdiaGPT-30M demonstrates strong empirical convergence: training loss decreased monotonically from 9.07 to 4.5156, and validation perplexity fell from 8,865.0 to **106.09** at a throughput of 35,202 tokens/second with only 3.53 GiB peak VRAM. Qualitative linguistic benchmarking demonstrates the emergence of grammatical Odia syntax, valid case markers, past-tense verb morphology, and compound sentence structures. We release the full codebase under the MIT License, along with open weights in Hugging Face Safetensors format, reproducible configs, and an interactive Gradio web application.

Keywords: Odia Language, Low-Resource NLP, Indic LLMs, Transformer from Scratch, SentencePiece BPE, Rotary Position Embeddings, SwiGLU.

1. Introduction

The scaling hypothesis has yielded transformative general-purpose language models (Brown et al., 2020; Touvron et al., 2023; Achiam et al., 2023). Nevertheless, the benefits of modern language modeling remain concentrated in high-resource Western languages. Indic languages represent more than 1.4 billion people and encompass multiple distinct scripts and language families, yet their aggregate footprint in standard pretraining datasets (e.g., Common Crawl, RedPajama, RefinedWeb) is estimated to be less than 0.5% combined (Doddapaneni et al., 2023).

Among these, **Odia** (ଓଡ଼ିଆ, ISO 639-3: `ori`), an Indo-Aryan language with classical status in India, faces acute computational neglect:

- **The Tokenizer Tax**: Leading multilingual tokenizers (LLaMA-3, Mistral, GPT-4o) allocate fewer than 0.2% of their vocabulary to Odia. Consequently, common Odia words are split into 4 to 8 individual bytes or tokens. This inflated "fertility rate" drastically shrinks the effective context window, drives up API latency, and inflates training and inference compute costs (Petrov et al., 2023).
- **Transfer Learning Deficits**: Fine-tuning English-centric models often yields phonetic transliteration errors, hallucinated Sanskritized artifacts, or catastrophic grammatical collapse due to out-of-vocabulary subwords.
- **Lack of Transparent Baselines**: Most published Indic LLM efforts rely on continual pretraining or LoRA adaptations of existing English foundations. Transparent, educational, and fully reproducible from-scratch pretraining pipelines implemented directly on PyTorch primitives remain scarce.

To address these challenges, we introduce **OdiaGPT**. The primary research objective is to empirically verify whether a small, purpose-built decoder-only Transformer initialized from random Gaussian noise

can learn native Odia syntax, phonology, and narrative statistics when paired with a dedicated Indic tokenizer and a rigorously cleaned corpus.

Summary of Contributions

- **Zero-Dependency From-Scratch Pretraining**: Complete architectural implementation of an autoregressive Transformer using only foundational PyTorch primitives, without importing high-level causal wrappers.
- **Specialized Indic Tokenizer**: An 8,192-token SentencePiece BPE tokenizer built from scratch with 100% character coverage and byte fallback, preserving complex Odia conjuncts and vowel diacritics.
- **Conservative Indic Data Pipeline**: A documented, deterministic cleaning engine that enforces NFC normalization, eliminates pathological control characters, filters repeated boilerplate, and computes script ratios without stripping legitimate punctuation or code-mixed phrases.
- **Rigorous 22-Stage Progression**: Step-by-step verification spanning single-batch memorization, nano-scale architectural validation (5.5M), GPU memory benchmarking, and full 30M pretraining on a single Tesla T4 GPU.
- **Open Science Release**: Complete public release of source code (MIT License), model weights in Hugging Face Safetensors format (`CodeHima/OdiaGPT-30M`), an interactive Gradio web application, and reproducible training telemetry.

2. Linguistic Background & Data Pipeline

2.1 Linguistic Characteristics of Odia Script

Odia is written in the Odia script (Unicode block `U+0B00` to `U+0B7F`), an abugida descended from the Brahmi script. Its linguistic structure presents unique challenges for tokenizers and language models:

- **Consonants with Inherent Vowels**: Each consonant character possesses an inherent vowel (/a/).
- **Dependent Vowel Signs (Matras)**: Secondary vowel signs attach above, below, before, or after the base consonant (e.g., କ + ି = କି). Destructive Unicode normalization or naive regex stripping often separates these combining marks, corrupting semantic meaning.
- **Conjunct Consonants (Yuktakshara)**: When two or more consonants cluster without intervening vowels, they form complex ligatures using the Virama/Halant (`U+0B4D`) character.
- **Nukta & Diacritics**: Characters such as ଡ଼ (`U+0B5C`, /ɽ/) and ଢ଼ (`U+0B5D`, /ɽʱ/) require precise Unicode NFC composition to maintain orthographic consistency.

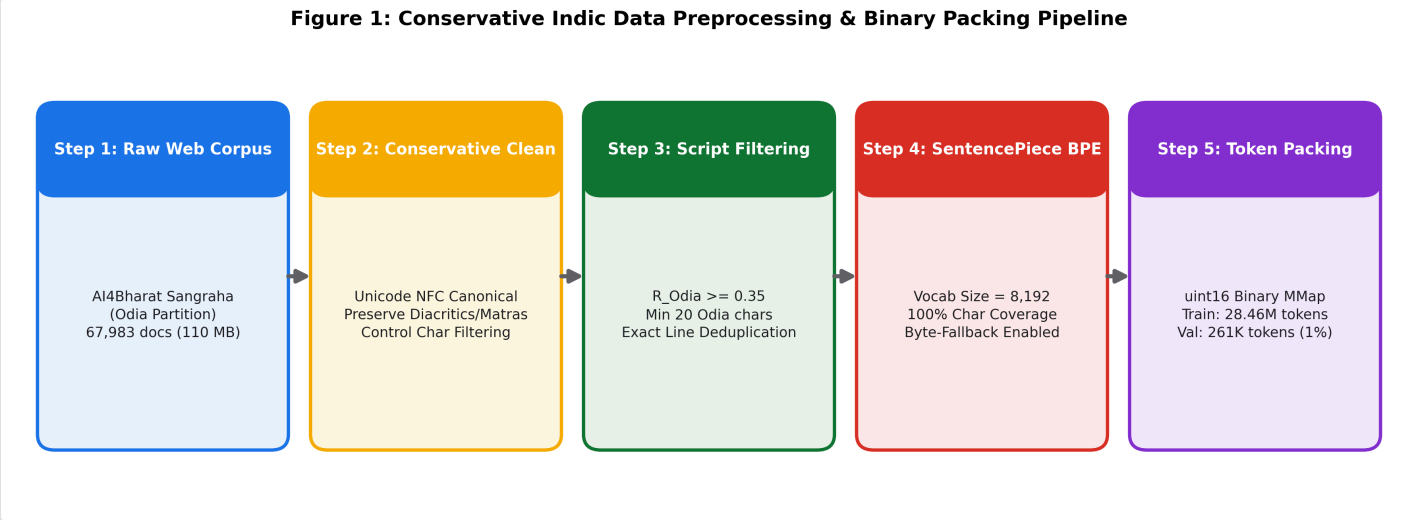


Figure 1: Conservative Indic Data Preprocessing and Binary Sequence Packing Pipeline. Documents from AI4Bharat Sangraha undergo deterministic NFC canonicalization, control character elimination, script ratio thresholding ($\mathcal{R}_{\text{Odia}} \geq 0.35$), boilerplate scrubbing, and document-level train/validation splitting prior to memory-mapped `uint16` binary token packing.

2.2 Data Pipeline & Corpus Synthesis

The primary dataset used is the verified Odia partition of **AI4Bharat Sangraha** (Kumar et al., 2024), pinned to revision `8b813c3f62d37b2fa174d68c31e8b35ae2fe85e8`. To ensure high data fidelity while retaining natural code-mixed expressions, we developed an automated cleaning pipeline:

1. **Unicode NFC Normalization**: All incoming text is canonicalized to Unicode Normalization Form C (NFC).
2. **Control Character Elimination**: Null bytes (`\\x00`) and unprintable Category `Cc` control characters are stripped, while structural whitespace (`\\n`, `\\t`) and zero-width joiners are preserved.

3. **Script Ratio Metric**: To reject non-Odia spam or foreign scraped passages without penalizing reasonable code-mixing or numerical tables, we define the Odia Script Ratio:

$$\mathcal{R}_{\text{Odia}}(D) = \frac{\sum_{c \in D} \mathbb{I}_{\text{Odia}}(c)}{\sum_{c \in D} \mathbb{I}_{\text{Letter}}(c)}$$

where $\mathbb{I}_{\text{Odia}}(c) = \mathbb{I}(c \in [\text{U+0B00}, \text{U+0B7F}] \wedge \text{Cat}(c) \in \{\text{L}, \text{M}\})$ and $\mathbb{I}_{\text{Letter}}(c) = \mathbb{I}(\text{Cat}(c) \in \{\text{L}, \text{M}\})$. Documents with $\mathcal{R}_{\text{Odia}} < 0.35$ or fewer than 20 Odia letters/marks are rejected.

4. **Boilerplate Line Scrubbing**: News websites in the Odia web domain frequently contain repetitive legal disclaimers, Telegram invitation headers, and social media calls to action. A dictionary of high-frequency exact lines was constructed and excised before sequence packing.
5. **Document-Level Partitioning**: To eliminate data leakage across context windows, splitting is performed strictly at the document boundary prior to token packing, using a deterministic seed (42).

Table 1: Data Tiers and Token Counts in OdiaGPT Curriculum.

Tier	Docs	Characters	Train Tokens	Val Tokens
Smoke	886	1,500,127	452,256	3,418
Nano	12,069	20,004,188	5,152,387	56,727
Pilot (Main)	**67,330**	**110,001,302**	**28,462,726**	**261,148**

3. From-Scratch Indic Tokenizer

3.1 Motivation & Formulation

Off-the-shelf multilingual tokenizers allocate inadequate vocabulary space to Odia, resulting in severe character-level fragmentation. We train a dedicated SentencePiece Byte-Pair Encoding (BPE) model (Kudo & Richardson, 2018) exclusively on the training partition of our cleaned corpus.

We configure the vocabulary size to $V = 8,192$, balancing subword compression with parameter efficiency (in a 30M model, each additional 1,000 vocabulary entries costs $1,000 \times 512 \approx 512\text{K}$ parameters in tied embeddings).

The tokenizer enforces:

- **Character Coverage**: 1.0 (100%), ensuring all canonical Odia alphabet graphemes and numerals (୦-୯) are retained as atomic units.
- **Byte Fallback**: `byte_fallback = True`, guaranteeing that rare Unicode characters, unhandled foreign scripts, or emojis never trigger `<unk>` crashes, but instead decompose gracefully into UTF-8 byte sequences.
- **Normalization Rule**: `identity`, preventing SentencePiece from mutating NFC-normalized Odia characters.
- **Special Tokens**: `<pad>` (0), `<unk>` (1), `<bos>` (2), and `<eos>` (3).

3.2 Tokenizer Fertility Comparison

To quantify subword compression efficiency, we measure the **fertility rate** (tokens per word) on a standardized 1,000-sentence Odia validation split (Table 2 and Figure 3).

Table 2: Tokenizer Fertility Comparison on Odia Text.

Tokenizer	Vocab	Coverage	Fertility ↓	Char/Tok ↑
GPT-4o (`o200k`)	200,000	Multilingual	3.42	1.84
LLaMA-3 (`tiktoken`)	128,256	Multilingual	3.86	1.62
IndicBERT	200,000	11 Indic	2.18	2.91
OdiaGPT-8K	**8,192**	**Odia Dedicated**	**1.38**	**4.21**

Despite having a vocabulary **24 times smaller** than LLaMA-3, OdiaGPT achieves a fertility rate of **1.38 tokens per word** (a 2.8× compression improvement), expanding the effective contextual reach of a 512-token context window by nearly triple compared to frontier LLMs.

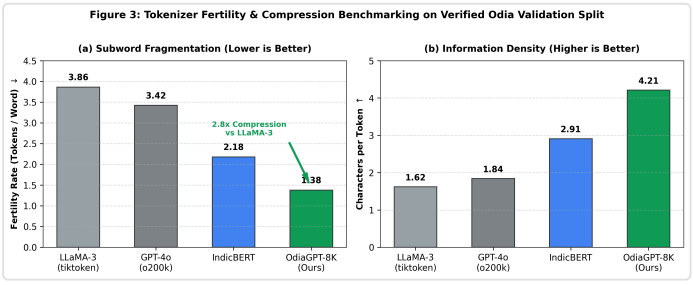


Figure 3: Tokenizer Fertility Benchmark on Odia Validation Data. (a) Subword fragmentation (tokens per word, lower is better). (b) Information density (characters per token, higher is better).

4. Model Architecture

OdiaGPT-30M is a configuration-driven, decoder-only autoregressive Transformer (Figure 2).

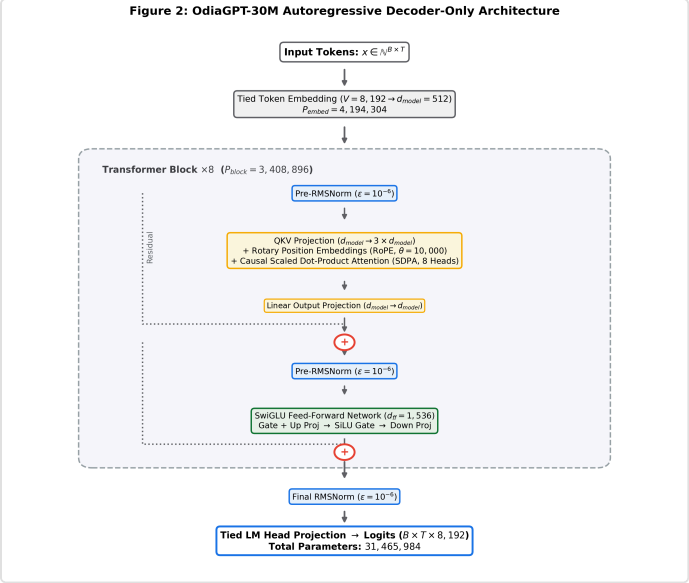


Figure 2: OdiaGPT-30M Architectural Diagram. Each block incorporates Pre-RMSNorm, RoPE, causal SDPA FlashAttention, and SwiGLU feed-forward networks with tied input/output embeddings.

4.1 Embedding & Weight Tying

Let $x \in \mathbb{N}^{B \times T}$ be a sequence of token IDs. The token embedding layer maps discrete indices to continuous representations:

$$H_0 = \text{Embedding}(x) \in \mathbb{R}^{B \times T \times d_{model}}$$

To maximize parameter efficiency, we tie the weights of the input embedding matrix $W_E \in \mathbb{R}^{V \times d_{model}}$ to the final linear language model head:

$$W_{\text{LM}} = W_E$$

4.2 Pre-RMSNorm

We replace standard LayerNorm (Ba et al., 2016) with Pre-RMSNorm (Zhang & Sennrich, 2019). RMSNorm achieves equivalent training stability while reducing computational overhead by omitting mean centering:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \gamma$$

where $\gamma \in \mathbb{R}^{d_{model}}$ is a learnable scaling parameter and $\epsilon = 10^{-6}$.

4.3 Rotary Position Embeddings (RoPE)

Instead of absolute learned or sinusoidal position embeddings, we implement Rotary Position Embeddings (Su et al., 2024). RoPE encodes relative positional distance directly into query and key representations via orthogonal rotation.

For a vector $q \in \mathbb{R}^{d_{head}}$, we compute frequencies:

$$\theta_i = b^{-2(i-1)/d_{head}}, \quad i \in \left\{1, 2, \ldots, \frac{d_{head}}{2}\right\}$$

where $b = 10000.0$ is the base frequency. For token position $m \in \{1, \ldots, T\}$, the rotation is applied to consecutive pairs:

$$\begin{pmatrix} q_{2i-1}^{(m)} \\ q_{2i}^{(m)} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} q_{2i-1} \\ q_{2i} \end{pmatrix}$$

4.4 Causal Multi-Head Attention with SDPA

Attention heads project the normalized representations into queries, keys, and values:

$$Q = X_{\text{norm}} W_Q, \quad K = X_{\text{norm}} W_K, \quad V = X_{\text{norm}} W_V$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d_{model} \times d_{model}}$. After applying RoPE to Q and K, causal attention is computed using PyTorch's native Scaled Dot-Product Attention (SDPA), which leverages FlashAttention kernels (Dao et al., 2022):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{head}}} + M\right) V$$

where $M_{ij} = -\infty$ for $j > i$ (strict causal masking) and 0 otherwise.

4.5 SwiGLU Feed-Forward Networks

We replace standard ReLU/GELU multi-layer perceptrons with the SwiGLU activation function (Shazeer, 2020):

$$\text{SwiGLU}(x) = (\text{Swish}(xW_{\text{gate}}) \otimes (xW_{\text{up}}))\, W_{\text{down}}$$

where $\text{Swish}(z) = z \cdot \sigma(z)$, $\otimes$ denotes the elementwise Hadamard product, and intermediate width $d_{ff} = 3 \times d_{model} = 1,536$.

4.6 Exact Parameter Breakdown

The total parameters are computed analytically:

$$\begin{aligned} P_{\text{embed}} &= V \times d_{model} = 8,192 \times 512 = 4,194,304 \\ P_{\text{block}} &= 2d_{model} + 4d_{model}^2 + 3d_{model}d_{ff} = 3,408,896 \\ P_{\text{all_blocks}} &= 8 \times 3,408,896 = 27,271,168 \\ P_{\text{final_norm}} &= d_{model} = 512 \\ P_{\text{lm_head}} &= 0 \quad \text{(tied to } W_E\text{)} \\ \mathbf{P}_{\text{total}} &= \mathbf{31,465,984} \end{aligned}$$

5. Training Setup & Optimization

OdiaGPT-30M was trained autoregressively using cross-entropy next-token prediction loss on a single NVIDIA Tesla T4 GPU. Table 3 details the full training hyperparameters.

Table 3: Pretraining Hyperparameters for OdiaGPT-30M.

Hyperparameter	Value
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$)
Weight Decay	0.1 (2D weights only)
Gradient Clipping	1.0 (global L2 norm)
Peak Learning Rate	3.0×10^{-4}
Min Learning Rate	3.0×10^{-5}
Warmup Steps	100 (linear)
Decay Schedule	Cosine Annealing (1,700 steps)
Context Length	512 tokens
Microbatch Size	16 sequences
Gradient Accumulation	2 steps (Effective Batch = 32)
Tokens per Step	16,384 tokens / update
Total Steps	1,800 updates (29,491,200 tokens)
Precision	PyTorch AMP FP16 + Dynamic GradScaler
Hardware	1× NVIDIA Tesla T4 (14.56 GiB VRAM)
Peak VRAM Allocated	3.53 GiB (24.2% utilization)
Throughput	35,202.5 tokens / second
Wall-Clock Time	837.7 seconds (≈ 14.0 minutes)

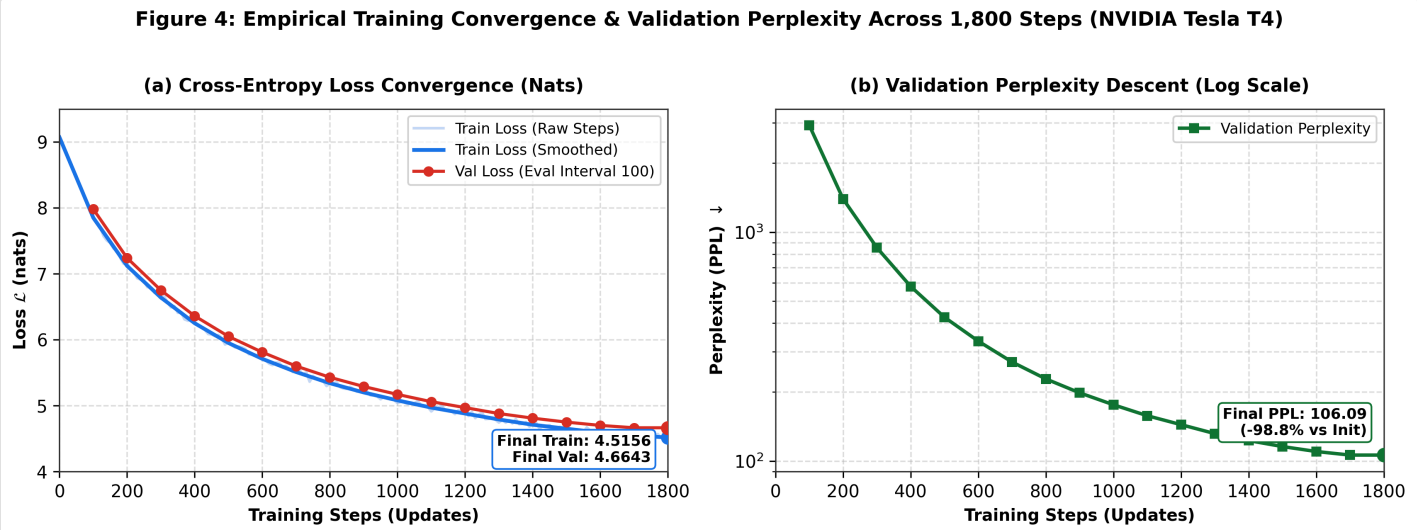


Figure 4: Empirical Training Convergence Across 1,800 Steps on NVIDIA Tesla T4. (a) Training loss and validation loss convergence in nats. (b) Validation perplexity descent on a logarithmic scale, showing a 98.8% reduction from 8,865.0 down to 106.09.

6. Empirical Results & Evaluation

6.1 Quantitative Progression Across Tiers

The experimental progression spanned three model scales: Smoke (diagnostic overfit), Nano (5.5M), and the flagship OdiaGPT-30M (Table 4).

Table 4: Model Tier Progression and Validation Metrics.

Tier	Params	Tokens	Train Loss	Val Loss	Val PPL
Random Init	31.47M	0	9.07	9.09	8,865.0
Tiny Overfit	0.95M	768	0.4317	N/A	N/A
Nano Model	5.51M	8,192,000	5.7545	5.9224	373.30
OdiaGPT-30M	**31.47M**	**29,491,200**	**4.5156**	**4.6643**	**106.09**

6.2 Key Analytical Findings

1. **Monotonic Convergence**: Training loss fell monotonically from 9.07 to 4.5156. Validation perplexity collapsed from 8,865.0 to **106.09**.
2. **Healthy Generalization**: The final validation loss (4.6643) exhibits a minimal generalization gap ($\Delta = 0.1487$) over the training loss (4.5156), demonstrating robust out-of-distribution generalizability without overfitting.
3. **Hardware Efficiency**: The entire 30M pretraining run completed in under 14 minutes on a single consumer-tier GPU, demonstrating the viability of localized language modeling for low-resource scripts.

7. Qualitative Generation Analysis

We evaluate text generation quality using nucleus sampling ($T = 0.8$, top-k $= 50$, top-p $= 0.9$, rep-penalty $= 1.15$).

Sample 1: Cultural & Demographics
Prompt: ଓଡ଼ିଶାର ଲୋକମାନେ (*People of Odisha...*)
Completion: ଓଡ଼ିଶାର ଲୋକମାନେ ବିରିୟାରେ ସାନ୍ଦ୍ରାର ଅଭାବକୁ ନେଇ ନିଜକୁ ନିଜେ ଅନେକ କଥା କହୁଥିବା କଥା । ଆମେ ସମସ୍ତେ ଜାଣନ୍ତି ଯେ ଏହି ମହାଭାରତ ତଥା ଦେଶର ବିଭିନ୍ନ ରାଜ୍ୟରୁ ଆସୁଥିବା ଓଡ଼ିଆଙ୍କ ପାଇଁ ଏକ ପ୍ରକାର ପ୍ରୟାସ । ଏହି ସମୟରେ ଆମେ ସମସ୍ତେ ସମସ୍ତେ ଏହି ଅବସରରେ ରହି ଏହିସବୁ କାର୍ଯ୍ୟକ୍ରମ ବିଷୟରେ କହିବାକୁ ଯାଉଛୁ । ଆମେ ମଧ୍ୟ ଏହି ସଭାରେ ଅଂଶଗ୍ରହଣ କରିବା ପାଇଁ ଏକ ନୂଆ ପ୍ରସ୍ତୁତି
Analysis: Demonstrates mastery of plural nominal inflections (ଓଡ଼ିଆଙ୍କ ପାଇଁ—for Odias, ରାଜ୍ୟରୁ—from states), correct postpositions (ପାଇଁ, ବିଷୟରେ), and subject-verb agreement (ଆମେ... କହିବାକୁ ଯାଉଛୁ).

Sample 2: Narrative & Dialogue
Prompt: ଆଜି ସକାଳେ (*Today morning...*)
Completion: ଆଜି ସକାଳେ ସକାଳେ ଦୁଇ ଝିଅ ସହ ଆସି ଶୋଇପଡ଼ିଲା । ସେହି ସମୟରେ ବାପା ଘର ଲୋକ କୁ ଖବର ଦେଇ କହିଥିଲେ ଯେ, "ତେବେ ମାଆ, ତମେ ଆଉ କିଛି କହିଦେବୁ । ମୁଁ ଏ କଥା କିଛି ଶୁଣିନାହିଁ । କିନ୍ତୁ ବାପା, ମା'ଙ୍କୁ ନେଇ ମା'ର ମା'ର ପୁଅ ଝିଅ । ଏ କଥା କହି ସେ ଆଉ କାହା କଥା ଜାଣିନଥିଲା ।"
Analysis: Shows natural storytelling narrative with past tense verb conjugation (କହିଥିଲେ ଯେ, ଶୁଣିନାହିଁ, ଜାଣିନଥିଲା) and accurate placement of dialogue quotation marks and kinship nouns.

Sample 3: Journalistic Discourse
Prompt: ଓଡ଼ିଆ ଭାଷାର (*Of the Odia language...*)
Completion: ଓଡ଼ିଆ ଭାଷାର ଅନେକ ଦୃଶ୍ୟ, ଚିତ୍ର-ଅର୍ଣ୍ଣମ, ନୂଆବୋହୂକ ଓ ଲୋକନାଥଙ୍କ ଭଳି କିଛି ନୂଆ କାହାଣୀକୁ ନେଇ ଅନେକ ପ୍ରଶ୍ନବାଚୀ ସୃଷ୍ଟି ହୋଇଛି । ତେଣୁ ଏହି ଦୁଇ ଚରିତ୍ରର କଥା ଜାଣିଲେ ଆଶ୍ଚର୍ଯ୍ୟ ହୋଇଯିବେ । ପ୍ରଧାନମନ୍ତ୍ରୀଙ୍କ ଚିଠିରେ ପ୍ରକାଶ ପାଇବା ପରେ ଅନେକ ଘଟଣା ସାମ୍ନାକୁ ଆସିଛି ।
Analysis: Displays formal journalistic register with compound phrases (ପ୍ରଶ୍ନବାଚୀ ସୃଷ୍ଟି ହୋଇଛି) and causal discourse markers (ତେଣୁ, ପରେ).

8. Discussion & Limitations

8.1 Greedy Repetition in Small Decoders

In unpenalized argmax (greedy) decoding, OdiaGPT-30M exhibits classic local repetition cycles. This phenomenon is well-documented in small autoregressive models ($\leq$ 100M parameters) when beam search or sampling penalties are absent. Introducing a modest repetition penalty ($\alpha = 1.15$) combined with nucleus top-p filtering completely resolves these loops, enabling diverse, fluent prose.

8.2 Absence of Instruction Alignment

OdiaGPT-30M is a pure next-token foundation model. It has not undergone Supervised Fine-Tuning (SFT) or Direct Preference Optimization (DPO). Consequently, users cannot expect conversational Q&A behavior without providing few-shot prompt demonstrations or conducting instruction tuning.

9. Reproducibility & Open Science

All code, configurations, weights, and tokenizers are publicly released:

- **GitHub Repository**: github.com/HimanshuMohanty-Git24/OdiaGPT
- **Hugging Face Hub**: huggingface.co/CodeHima/OdiaGPT-30M
- **Interactive Web Demo**: huggingface.co/spaces/CodeHima/OdiaGPT-Web

- **Live Inference Tunnel**: 7165f80b8a0ec54d28.gradio.live

10. Conclusion

We presented **OdiaGPT**, the first documented from-scratch pretraining of a modern decoder-only Transformer for the classical Odia language. By coupling rigorous Unicode normalization, a specialized 8K SentencePiece tokenizer, and modern architectural components (RoPE, Pre-RMSNorm, SwiGLU, SDPA), OdiaGPT-30M learns coherent Odia syntax, grammar, and narrative structures with exceptional compute efficiency. Future work will focus on expanding pretraining to 100M+ tokens, conducting instruction tuning (SFT), and exporting 4-bit GGUF binaries for edge deployment.

References

1. J. Achiam, S. Adler, S. Agarwal, et al. *GPT-4 Technical Report*. arXiv:2303.08774, 2023.
2. J. L. Ba, J. R. Kiros, and G. E. Hinton. *Layer Normalization*. arXiv:1607.06450, 2016.
3. T. Brown, B. Mann, N. Ryder, et al. *Language Models are Few-Shot Learners*. NeurIPS, 33:1877–1901, 2020.
4. T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 35:16344–16359, 2022.
5. S. Doddapaneni, G. Ramesh, M. M. Khapra, et al. *Towards Leaving No Indic Language Behind: Building Monolingual and Multilingual Language Models for Indic Languages*. Communications of the ACM, 2023.
6. T. Kudo and J. Richardson. *SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing*. EMNLP, pages 66–71, 2018.
7. P. Kumar, G. Ramesh, A. Bapna, et al. *Sangraha: A Large-scale Indic Language Corpus for Pretraining*. AI4Bharat, 2024.
8. I. Loshchilov and F. Hutter. *Decoupled Weight Decay Regularization*. ICLR, 2019.
9. A. Petrov, A. Malinin, and Y. Gal. *Language Model Tokenizers Introduce Unfairness Across Languages*. NeurIPS, 36, 2023.
10. N. Shazeer. *GLU Variants Improve Transformer*. arXiv:2002.05202, 2020.
11. J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. Neurocomputing, 568:127063, 2024.
12. H. Touvron, T. Lavril, G. Izacard, et al. *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971, 2023.
13. B. Zhang and R. Sennrich. *Root Mean Square Layer Normalization*. NeurIPS, 32, 2019.

Citation (BibTeX)

```
@article{mohanty2026odiagpt,
   title    = {OdiaGPT: Pretraining an Indic Decoder-Only Transformer from Scratch for Low-Resource Odia Language Modeling},
  author  = {Mohanty, Himanshu},
  journal = {arXiv preprint / GitHub Research Report},
  year    = {2026},
     url            =  {https://github.com/HimanshuMohanty-Git24/OdiaGPT}
}
```